

Assignment No	: 3
Roll No	: 381
Batch	: E4
PRN No	: 202502040032
Title of the Assignment	: Multithreading

- a) Implement a program to Create thread, scheduler, thread methods all
- b) Program to implement thread using the runnable interface and creating multiple threads
- c) Write a program to priorities of thread, Threadpool, and thread group.

Solution : Smart car parking system .

Project Structure :

```

src
├── main
│   └── MainApp.java
├── service
│   └── ParkingLot.java
└── threads
    ├── MonitorTask.java
    └── VehicleTask.java

```

main/ MainApp.java

```

package main;

import service.ParkingLot;
import threads.MonitorTask;
import threads.VehicleTask;

import java.util.Scanner;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class MainApp {

    public static void main(String[] args) throws InterruptedException {

        Scanner sc = new Scanner(System.in);

        System.out.println("\n=====");
        System.out.println("    SMART CAR PARKING SYSTEM    ");
        System.out.println("=====\\n");

        System.out.print("Enter Total Parking Slots: ");
        int slots = sc.nextInt();

        System.out.print("Enter Number of Vehicles: ");
        int vehicleCount = sc.nextInt();

        int[] times = new int[vehicleCount];

```

```
for (int i = 0; i < vehicleCount; i++) {
    System.out.print("Enter parking time (ms) for Car-" + (i + 1) + ": ");
    times[i] = sc.nextInt();
}

ParkingLot parkingLot = new ParkingLot(slots);

System.out.println("\n===== SIMULATION STARTED =====\n");

MonitorTask monitor = new MonitorTask(parkingLot);
monitor.setDaemon(true);
monitor.start();

ExecutorService pool = Executors.newFixedThreadPool(3);

for (int i = 0; i < vehicleCount; i++) {
    pool.execute(new VehicleTask(parkingLot, "Car-" + (i + 1), times[i]));
}

pool.shutdown();

while (!pool.isTerminated()) {
    Thread.sleep(1000);
}

System.out.println("\n===== SIMULATION COMPLETED =====\n");

sc.close();
}
}
```

service/ ParkingLot.java

```
package service;

public class ParkingLot {

    private int totalSlots;
    private int availableSlots;

    public ParkingLot(int slots) {
        this.totalSlots = slots;
        this.availableSlots = slots;
    }

    public synchronized void enterParking(String carName) {

        Thread t = Thread.currentThread();

        while (availableSlots == 0) {
            try {
                System.out.println("\n[WAITING] " + carName);
                System.out.println("Thread: " + t.getName() +
                    " | ID: " + t.getId() +
                    " | Priority: " + t.getPriority());
                wait();
            } catch (InterruptedException e) {
            }
        }

        availableSlots--;
    }
}
```

```

        System.out.println("\n-----");
        System.out.println("CAR ENTERED PARKING");
        System.out.println("-----");
        System.out.println("Car Name      : " + carName);
        System.out.println("Thread Name   : " + t.getName());
        System.out.println("Thread ID     : " + t.getId());
        System.out.println("Thread Priority: " + t.getPriority());
        System.out.println("Total Slots   : " + totalSlots);
        System.out.println("Available Slots: " + availableSlots);
        System.out.println("-----");

        Thread.yield();
    }

    public synchronized void exitParking(String carName) {

        Thread t = Thread.currentThread();

        availableSlots++;

        System.out.println("\n-----");
        System.out.println("CAR EXITED PARKING");
        System.out.println("-----");
        System.out.println("Car Name      : " + carName);
        System.out.println("Thread Name   : " + t.getName());
        System.out.println("Thread ID     : " + t.getId());
        System.out.println("Available Slots: " + availableSlots);
        System.out.println("-----\n");

        notifyAll();
    }

    public synchronized int getAvailableSlots() {
        return availableSlots;
    }
}

```

threads/ MonitorTask.java

```

package threads;

import service.ParkingLot;

public class MonitorTask extends Thread {

    private ParkingLot parkingLot;

    public MonitorTask(ParkingLot parkingLot) {
        this.parkingLot = parkingLot;
        setName("Monitor-Thread");
        setPriority(Thread.MAX_PRIORITY);
    }

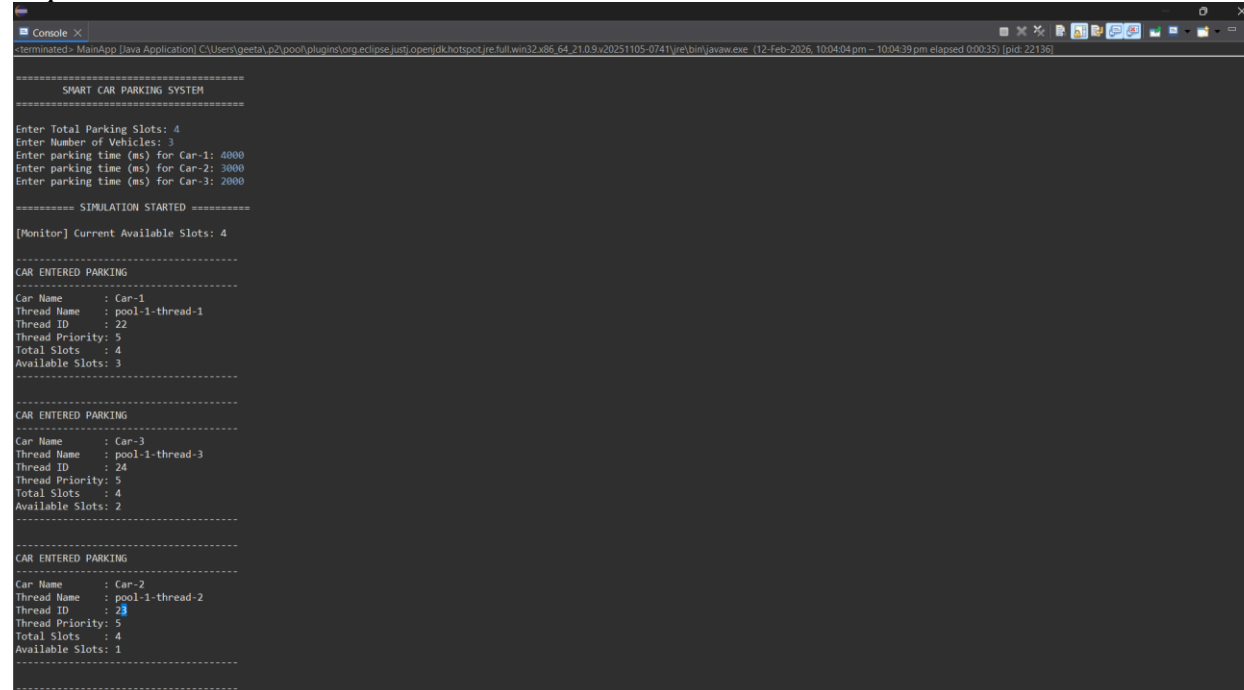
    public void run() {
        while (true) {
            System.out.println("[Monitor] Current Available Slots: "
                + parkingLot.getAvailableSlots());
            try {
                Thread.sleep(4000);
            } catch (InterruptedException e) {
                break;
            }
        }
    }
}

```

```
}  
}  
}
```

threads/ VehicleTask.java

```
package threads;  
  
import service.ParkingLot;  
  
public class VehicleTask implements Runnable {  
  
    private ParkingLot parkingLot;  
    private String carName;  
    private int parkingTime;  
  
    public VehicleTask(ParkingLot parkingLot, String carName, int parkingTime) {  
        this.parkingLot = parkingLot;  
        this.carName = carName;  
        this.parkingTime = parkingTime;  
    }  
  
    public void run() {  
  
        parkingLot.enterParking(carName);  
  
        try {  
            Thread.sleep(parkingTime);  
        } catch (InterruptedException e) {  
        }  
  
        parkingLot.exitParking(carName);  
    }  
}
```

Output :

```
Console x  
-terminated- MainApp [Java Application] C:\Users\geeta\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.9\jre\bin\java.exe. (12-Feb-2026, 10:04:04 pm - 10:04:39 pm elapsed 000:35) [pid: 22136]  
  
===== SMART CAR PARKING SYSTEM =====  
  
Enter Total Parking Slots: 4  
Enter Number of Vehicles: 3  
Enter parking time (ms) for Car-1: 4000  
Enter parking time (ms) for Car-2: 3000  
Enter parking time (ms) for Car-3: 2000  
  
===== SIMULATION STARTED =====  
  
[Monitor] Current Available Slots: 4  
  
-----  
CAR ENTERED PARKING  
-----  
Car Name      : Car-1  
Thread Name   : pool-1-thread-1  
Thread ID    : 22  
Thread Priority: 5  
Total Slots   : 4  
Available Slots: 3  
-----  
  
CAR ENTERED PARKING  
-----  
Car Name      : Car-3  
Thread Name   : pool-1-thread-3  
Thread ID    : 24  
Thread Priority: 5  
Total Slots   : 4  
Available Slots: 2  
-----  
  
CAR ENTERED PARKING  
-----  
Car Name      : Car-2  
Thread Name   : pool-1-thread-2  
Thread ID    : 21  
Thread Priority: 5  
Total Slots   : 4  
Available Slots: 1  
-----  
-----
```

```
terminated: MainApp [Java Application] C:\Users\geeta\p2\pool\plugins\org.eclipse.justi.openjdk hotspot.jre-full\win32\x86_64_21.0.9\20251105-0741\jre\bin\java.exe (12-Feb-2026, 10:04:04 pm - 10:04:39 pm elapsed 0:00:35) [pid: 22136]
Total Slots: 4
Available Slots: 2

-----
CAR ENTERED PARKING
-----
Car Name      : Car-2
Thread Name   : pool-1-thread-2
Thread ID    : 23
Thread Priority: 5
Total Slots  : 4
Available Slots: 1
-----

CAR EXITED PARKING
-----
Car Name      : Car-3
Thread Name   : pool-1-thread-3
Thread ID    : 24
Available Slots: 2
-----

CAR EXITED PARKING
-----
Car Name      : Car-2
Thread Name   : pool-1-thread-2
Thread ID    : 23
Available Slots: 3
-----

[Monitor] Current Available Slots: 3
-----

CAR EXITED PARKING
-----
Car Name      : Car-1
Thread Name   : pool-1-thread-1
Thread ID    : 22
Available Slots: 4
-----

***** SIMULATION COMPLETED *****
```

Submitted by,
Uddhav Ramprasad Tour